Job Shop Scheduling Problem
PROGETTO
Laboratorio di Intelligenza Artificiale

Carolina Crespi

9 Novembre 2026

1 Job Shop Scheduling Problem – JSP

Il *Job Shop Scheduling Problem* (JSP) è un problema di ottimizzazione combinatoria, classificato come NP-hard, ed è uno dei problemi di scheduling più studiati in informatica e ricerca operativa.

Intuitivamente, il problema modella la situazione in cui un insieme di lavori (*job*) deve essere eseguito su un insieme di macchine. Ogni job è composto da una sequenza di operazioni, ciascuna delle quali deve essere eseguita su una macchina specifica per un tempo prefissato. Si vuole stabilire l'istante di inizio di ciascuna operazione in modo che:

- le operazioni di uno stesso job siano eseguite nell'ordine prestabilito (*vincolo di precedenza*);
- ciascuna macchina esegua al più un'operazione alla volta (*vincolo di capacità*);
- il tempo di completamento dell'ultimo job, detto *makespan*, sia il più piccolo possibile.

La difficoltà del problema non risiede nei singoli vincoli, ma nella loro interazione: decidere l'ordine in cui le operazioni vengono eseguite su una macchina influenza i tempi di attesa dei job sulle macchine successive, generando dipendenze a catena. Alcuni esempi applicativi:

- la pianificazione di una linea di produzione manifatturiera, in cui ogni prodotto attraversa reparti diversi (taglio, assemblaggio, verniciatura) in un ordine specifico;
- lo scheduling di task su unità di elaborazione condivise (CPU, GPU, dispositivi di I/O);
- la pianificazione di esami diagnostici in ambito ospedaliero, in cui pazienti diversi devono accedere agli stessi macchinari secondo percorsi differenti.

Formalmente, siano dati:

- un insieme di n job $\mathcal{J} = \{J_1, \ldots, J_n\}$;
- un insieme di m macchine $\mathcal{M} = \{M_1, \ldots, M_m\}$;
- per ciascun job J_i, una sequenza ordinata di m operazioni $O_{i,1}, O_{i,2}, \ldots, O_{i,m}$, in cui l'operazione $O_{i,k}$ deve essere eseguita sulla macchina $\mu_{i,k} \in \mathcal{M}$ per un tempo $p_{i,k} > 0$. Ogni job visita ciascuna macchina esattamente una volta.

Introducendo le variabili $s_{i,k} \geq 0$, che rappresentano l'istante di inizio dell'operazione $O_{i,k}$, e la variabile $C_{\max}$, che rappresenta il makespan, il problema si formula come:

$$\min \; C_{\max} \tag{1}$$

soggetta ai vincoli:

$$s_{i,k+1} \geq s_{i,k} + p_{i,k} \qquad \forall\, i = 1, \ldots, n, \quad k = 1, \ldots, m-1, \tag{2}$$

$$s_{i,k} \geq s_{j,l} + p_{j,l} \quad \vee \quad s_{j,l} \geq s_{i,k} + p_{i,k} \qquad \forall\, (i,k), (j,l) \text{ con } i \neq j \text{ e } \mu_{i,k} = \mu_{j,l}, \tag{3}$$

$$C_{\max} \geq s_{i,m} + p_{i,m} \qquad \forall\, i = 1, \ldots, n, \tag{4}$$

$$s_{i,k} \geq 0 \qquad \forall\, i, k. \tag{5}$$

I vincoli (2) impongono la precedenza tra operazioni dello stesso job; i vincoli disgiuntivi (3) garantiscono che due operazioni assegnate alla stessa macchina non si sovrappongano nel tempo (una delle due deve terminare prima che l'altra inizi); i vincoli (4) definiscono il makespan come il massimo tra i tempi di completamento dei job.

Una soluzione può quindi essere vista come la scelta, per ciascuna macchina, di un ordine di esecuzione delle operazioni a essa assegnate: fissato tale ordine, gli istanti di inizio si ottengono anticipando ogni operazione il più possibile compatibilmente con i vincoli. Lo spazio di ricerca cresce come $(n!)^m$, sebbene non tutte le combinazioni di ordinamenti siano ammissibili.

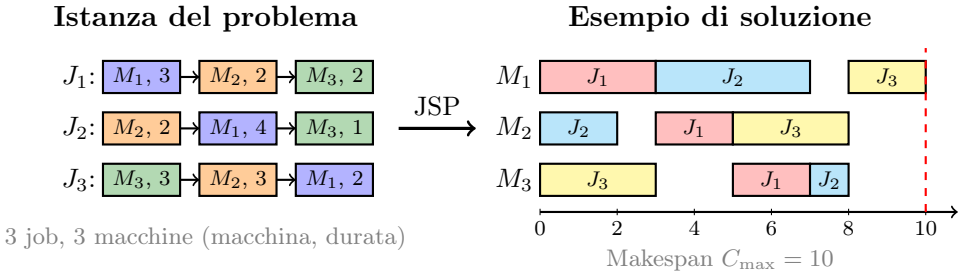


Figura 1: A sinistra: un'istanza illustrativa con 3 job e 3 macchine; per ciascun job è indicata la sequenza ordinata delle operazioni, con la macchina richiesta e la durata. A destra: un esempio di soluzione ammissibile rappresentata come diagramma di Gantt, in cui ogni riga corrisponde a una macchina e ogni blocco a un'operazione; la linea tratteggiata indica il makespan. La figura ha scopo unicamente illustrativo e non corrisponde all'istanza considerata nel progetto.

2 Istanza del Problema

Nel progetto è considerata una singola istanza del JSP composta da $n = 4$ job e $m = 4$ macchine, per un totale di 16 operazioni.

I dati dell'istanza sono forniti nel file allegato alla presente traccia:

- `jobs.txt`: la prima riga contiene il numero di job n e il numero di macchine m; ciascuna delle n righe successive descrive un job, elencando in ordine di esecuzione le sue m operazioni come coppie *(macchina, durata)*. Le macchine sono indicizzate da 1 a m, coerentemente con la notazione $M_1, \ldots, M_m$.

Il valore ottimo della funzione obiettivo è noto ed è pari a $C^*_{\max} = 32$. Tale valore può essere utilizzato per verificare la correttezza delle soluzioni ottenute.

3 Protocollo Sperimentale

Il protocollo sperimentale varia in funzione della natura dell'algoritmo scelto.

Algoritmi deterministici (Branch and Bound, Greedy SPT)

Per gli algoritmi deterministici è richiesta **una sola esecuzione**. Nella relazione finale devono essere riportati:

- il valore del makespan ottenuto;
- lo schedule ottenuto, ovvero l'istante di inizio di ciascuna operazione, accompagnato dal relativo diagramma di Gantt;
- la verifica dell'ammissibilità della soluzione (vincoli di precedenza e di capacità rispettati);
- per il Branch and Bound: la strategia di branching, il lower bound utilizzato, la strategia di esplorazione dell'albero e il numero di nodi esplorati;
- per il Greedy SPT: la regola adottata in caso di parità e lo scostamento percentuale del makespan ottenuto rispetto al valore ottimo $C^*_{\max}$.

Algoritmi stocastici (Tabu Search, Algoritmo Genetico)

Per gli algoritmi stocastici sono richieste **5 run indipendenti**, utilizzando seed diversi. Ogni run è governata dal seguente criterio di arresto:

- il raggiungimento del valore ottimo noto $C^*_{\max} = 32$, oppure
- il raggiungimento di un numero massimo di **1000 valutazioni** della funzione obiettivo.

Una *valutazione della funzione obiettivo* corrisponde alla costruzione dello schedule associato a una soluzione completa e al calcolo del suo makespan.

Nella relazione finale devono essere riportati:

- il valore del makespan ottenuto in ciascuna run;
- lo schedule e il diagramma di Gantt della run migliore;
- la verifica dell'ammissibilità della soluzione;
- la media dei valori finali sulle 5 run;
- un breve commento sul comportamento dell'algoritmo adottato.

4 Algoritmo da Implementare

Per la Tabu Search e l'Algoritmo Genetico, una soluzione può essere rappresentata come una permutazione delle 16 operazioni, letta come lista di priorità: scorrendo la lista, si schedula ogni volta la prima operazione il cui predecessore nello stesso job è già stato schedulato. In questo modo ogni permutazione corrisponde a uno schedule ammissibile.

Il candidato è invitato a scegliere e implementare uno dei seguenti algoritmi:

1. **Branch and Bound (BnB)**, che costruisce lo schedule un'operazione alla volta, scegliendo a ogni livello dell'albero quale operazione schedulare, e pota i nodi il cui lower bound non è migliore della miglior soluzione trovata. Devono essere descritti la strategia di branching, il lower bound utilizzato e la strategia di esplorazione dell'albero (Depth First, Best Bound First o mista).
2. **Algoritmo Greedy con regola Shortest Processing Time (SPT)**, che a ogni passo schedula, tra le operazioni disponibili, quella con tempo di esecuzione minore, avviandola al primo istante in cui sono liberi sia il job sia la macchina. La gestione dei casi di parità deve essere dichiarata nella relazione.

3. **Tabu Search (TS)**, con soluzione iniziale casuale, vicinato basato su un operatore di mossa per permutazioni (swap, insertion o inversion), tabu list contenente le mosse inverse a quelle eseguite di recente, tabu tenure (statica, dinamica o adattiva) e criterio di aspirazione a scelta del candidato. Tutti i parametri devono essere dichiarati e motivati nella relazione.

4. **Algoritmo Genetico (GA)**, con codifica a permutazione, crossover per permutazioni (ad esempio Order Crossover, Partially Mapped Crossover o Cycle Crossover), mutazione swap, insertion o inversion, selezione (roulette wheel, tournament o rank-based) e strategia di replacement a scelta del candidato. Tutti i parametri (dimensione della popolazione, p_c, p_m) devono essere dichiarati e motivati nella relazione.

5 Note Finali al Progetto

La relazione finale deve essere redatta in LaTeX, con una lunghezza minima di 4 pagine e massima di 12 pagine.

È fortemente sconsigliata l'inclusione di codice implementato, mentre è fortemente consigliato l'uso di pseudo-codice e diagrammi esplicativi.

La relazione deve illustrare in modo chiaro:

- l'algoritmo sviluppato e le sue caratteristiche principali;
- le motivazioni delle scelte progettuali effettuate;
- eventuali elementi di novità e originalità introdotti;
- un'analisi critica dei risultati ottenuti;
- considerazioni finali sul comportamento dell'algoritmo.

La valutazione finale si baserà principalmente su:

- correttezza dell'implementazione e rispetto dei vincoli;
- qualità dei risultati ottenuti;
- qualità espositiva della relazione;
- completezza e chiarezza dell'analisi.

Consegna: il progetto (codice sorgente) e la relazione finale (`.pdf` e `.tex`) devono essere inviati via email all'indirizzo `carolina.crespi@unict.it` entro le ore 14:00 del **23 novembre 2026**. Il candidato è invitato a comunicare la scelta dell'algoritmo da sviluppare entro le ore 16:00 del **11 novembre 2026**.